Online Compiler Execution Results

Python 3 Environment | C/Python Algorithms

Q1_Knapsack.py - 0/1 Knapsack

```python
# Q1. 0/1 Knapsack Optimization
def knapsack_01(W, weights, values, n):
    dp = [[0] * (W + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]

print("Maximum Profit =", knapsack_01(50, [10, 20, 30], [60, 100, 120], 3))
```

```
user@simats-compiler:~/algorithms$ python Q1_Knapsack.py
Maximum Profit = 220
```

Q2_Unbounded.py - Unbounded Knapsack

```python
# Q2. Unbounded Knapsack
def unbounded_knapsack(W, weights, values, n):
    dp = [0] * (W + 1)
    for i in range(W + 1):
        for j in range(n):
            if weights[j] <= i:
                dp[i] = max(dp[i], dp[i - weights[j]] + values[j])
    return dp[W]

print("Maximum Profit =", unbounded_knapsack(100, [5, 10, 15], [10, 30, 20], 3))
```

```
user@simats-compiler:~/algorithms$ python Q2_Unbounded.py
Maximum Profit = 300
```

Q3_LCS.py - Longest Common Subsequence

```python
# Q3. Longest Common Subsequence (LCS)
def lcs(X, Y):
    m, n = len(X), len(Y)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

print("LCS Length =", lcs("ABCDGH", "AEDFHR"))
```

```
user@simats-compiler:~/algorithms$ python Q3_LCS.py
LCS Length = 3
```

Q4_LIS.py - Longest Increasing Subsequence

```python
# Q4. Longest Increasing Subsequence (LIS)
def lis(arr):
    n = len(arr)
    dp = [1] * n
    for i in range(1, n):
        for j in range(0, i):
            if arr[i] > arr[j] and dp[i] < dp[j] + 1:
                dp[i] = dp[j] + 1
    return max(dp)

print("LIS Length =", lis([10, 22, 9, 33, 21, 50]))
```

```
user@simats-compiler:~/algorithms$ python Q4_LIS.py
LIS Length = 4
```

Q5_CoinChange.py - Minimum Coins

```python
# Q5. Coin Change Problem (Minimum Coins)
def min_coins(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for coin in coins:
            if i - coin >= 0:
                dp[i] = min(dp[i], dp[i - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

print("Minimum Coins =", min_coins([1, 2, 5], 11))
```

```
user@simats-compiler:~/algorithms$ python Q5_CoinChange.py
Minimum Coins = 3
```